AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Smart Contract Audit

Axcess Finance

Confidential - Prepared for the Axcess Engineering Team

This report presents the findings of a smart contract security audit conducted against Axcess Loan Vaults. Findings were identified through manual review of the contract source by the CredShields' security team.

0	0	11	6	0	17
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

M001 `repay()` Pulls the Full Amount With No Refund of Overpayment — MEDIUM

M002 Lender Revocation Permanently Strands That Lender's Recovery Slice As Dead Capital — MEDIUM

M003 redeem() Pays Only msg.sender, Permanently Stranding Claims And Vault Liquidity For Non-Receiving Lenders — MEDIUM

M004 fundRedemptions() Leaves The Delinquency Cash Watermark Stale, Disabling The Anti-Donation Clamp — MEDIUM

M005 Protocol Fee Escapes Its Stated Subordination Inside The Termination Epoch — MEDIUM

M006 fundRedemptions() Retires Principal At The Pool Average, Causing The Capacity Cap To Drift From Actual Principal — MEDIUM

M007 Deposit And Borrow Gates Omit Scheduled Redemption Liabilities, Letting New Lender Capital Fund Queued Exits — MEDIUM

M008 Interest Compounds Per Segment, So Realised APR Exceeds The Configured Rate And Is Poke-Dependent — MEDIUM

M009 A Lender Blacklisted By The Loan Asset Cannot Be Paid, Stranding Their Recovery Slice As Dead Capital — MEDIUM

M010 The Delinquency Penalty Is Charged On The Whole Pool, So A One-Unit Shortfall Bills Penalty APR On Every Lender's Principal — MEDIUM

M011 Withdrawal Pause Removes The Only Cure For A Matured Redemption Cohort While The Unpausable Penalty Machine Keeps Billing The Whole Pool — MEDIUM

L001 Funded-Lender Redemptions Can Be Frozen After Termination — LOW

L002 `redeem()` Does Not Enforce the Registry Freeze That `claimRecovery()` Enforces — LOW

L003 Terminated Vaults Permanently Lock Cash Held Above The Frozen Book — LOW

L004 ERC-4626 max* Views Contradict Execution — LOW

L005 Deposit Gate Validates The Receiver But Never The Payer — LOW

L006 Unbounded Per-Epoch Accrual Loop Scales Gas With Elapsed Time — LOW

– Lead Reviewers & Signatories

/ 01: THE PICTURE AT A GLANCE

Executive Summary

The audit of Axcess smart contracts identified **17 findings** in total: **11 medium**, **6 low**, **0 high**, **0 critical**, and **0 informational**. The current retest posture shows **12 fixed** and **5 acknowledged** out of 17 findings.

There were no high or critical issues reported, so the main priorities are the medium-severity findings. These include repayment and redemption logic problems, lender recovery and blacklist handling issues, stale delinquency and cap accounting, and several post-operation checks that allow protocol state to drift or misapply fees and penalties. Together, these issues can strand funds, misstate balances, or let the protocol behave inconsistently under edge cases.

The low-severity findings mainly cover access control gaps, frozen redemption flows, locked cash in terminated vaults, ERC-4626 view and execution mismatches, and a gas-heavy accrual loop. While lower in severity, they still merit remediation to reduce operational risk and improve consistency across the vault lifecycle.

Audited scope

src/LoanVault.sol
src/LoanVaultBase.sol
src/LoanVaultDeposits.sol
src/LoanVaultWithdrawals.sol
src/LoanVaultBorrower.sol
src/VaultFactory.sol
src/RoleManager.sol
src/VaultLenderRegistry.sol
src/VaultAccountingLib.sol
src/WithdrawalQueueLibrary.sol
src/DualPausableUpgradeable.sol

Retested scope

https://github.com/Axcess-Finance/axcess-contract/tree/c85a8410afd9359441ec027793479af457baeaf5

Assessment overview

This engagement was a smart contract security audit of Axcess, focused on reviewing the on-chain business logic, access controls, state transitions, and edge case handling across the protocol's core flows. The assessment emphasized how funds move through lending, redemption, termination, and delinquency paths, with particular attention to whether the contract behavior remained consistent under pause, blacklist, revocation, and maturity conditions. The findings were then validated against their operational impact, with retest results showing that most issues had already been fixed, while a smaller set remained acknowledged at the time of review.

Overall, the review identified 17 issues, all in the low to medium range, with no high or critical severity findings. The dominant theme was **redemption and recovery path correctness**, especially around how lender balances and termination-state assets are handled. Several medium findings describe ways funds can become stranded or misdirected, including **redeem() Pays Only msg.sender, Permanently Stranding Claims And Vault Liquidity For Non-Receiving Lenders**, **A Lender Blacklisted By The Loan Asset Cannot Be Paid, Stranding Their Recovery Slice As Dead Capital**, and **Lender Revocation Permanently Strands That Lender's Recovery Slice As Dead Capital**. Related termination and recovery concerns also appear in **Funded-Lender Redemptions Can Be Frozen After Termination**, **Terminated Vaults Permanently Lock Cash Held Above The Frozen Book**, and **Withdrawal Pause Removes The Only Cure For A Matured Redemption Cohort While The Unpausable Penalty Machine Keeps Billing The Whole Pool**, all of which indicate that once the vault enters certain end-of-life or paused states, some recovery paths can become unavailable or incomplete.

A second major theme was **accounting and protocol-state drift**, where internal bookkeeping does not fully track the real economic state of the pool. Notable examples include **fundRedemptions() Leaves The Delinquency Cash Watermark Stale, Disabling The Anti-Donation Clamp**, **fundRedemptions() Retires Principal At The Pool Average, Causing The Capacity Cap To Drift From Actual Principal**, **Deposit And Borrow Gates Omit Scheduled Redemption Liabilities, Letting New Lender Capital Fund Queued Exits**, and **Protocol Fee Escapes Its Stated Subordination Inside The Termination Epoch**. These findings suggest that some checks and caps are computed from stale or incomplete state, which can distort fee priority, capacity enforcement, and eligibility decisions.

The remaining medium issues center on **logic and penalty calculation defects**. **repay() Pulls the Full Amount With No Refund of Overpayment** and **Interest Compounds Per Segment, So Realised APR Exceeds The Configured Rate And Is Poke-Dependent** both indicate that the observed behavior can diverge from intended economic expectations. Similarly, **The Delinquency Penalty Is Charged On The Whole Pool, So A One-Unit Shortfall Bills Penalty APR On Every Lender's Principal** shows that a localized shortfall can propagate into a broader and unintended penalty effect across the pool. The low-severity findings were concentrated in access-control and operational robustness areas, including **Deposit Gate Validates The Receiver But Never The Payer**, **redeem() Does Not Enforce the Registry Freeze That claimRecovery() Enforces**, **ERC-4626 max Views Contradict Execution**, **Unbounded Per-Epoch Accrual Loop Scales Gas With Elapsed Time**, and **Ether Locked Due to Missing Withdrawal Path**. Taken together, the findings portray a protocol whose main risks are not catastrophic exploits, but rather correctness issues that can strand value, misstate balances, or cause adverse behavior under specific lifecycle conditions.

ENGAGEMENT DETAILS

CLIENT	Axcess Finance
ASSESSMENT TYPE	Smart Contract Security Audit
AUDIT	Axcess Loan Vaults
REPORT DATE	18 Aug 2026
START DATE	14 Aug 2026
END DATE	20 Aug 2026
RETEST DATE	31 Aug 2026
TOTAL FINDINGS	17
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

0	0	11	6	0
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
M001	`repay()` Pulls the Full Amount With No Refund of Overpayment	MEDIUM	Business Logic (SC02-LogicErrors)	✓ FIXED
M002	Lender Revocation Permanently Strands That Lender's Recovery Slice As Dead Capital	MEDIUM	Privileged Role Mismanagement (SCWE-017)	ACKNOWLEDGED
M003	redeem() Pays Only msg.sender, Permanently Stranding Claims And Vault Liquidity For Non-Receiving Lenders	MEDIUM	Transfers to Addresses That Cannot Receive Funds (SCWE-149)	✓ FIXED
M004	fundRedemptions() Leaves The Delinquency Cash Watermark Stale, Disabling The Anti-Donation Clamp	MEDIUM	Failure to Handle Edge Cases (SCWE-083)	✓ FIXED
M005	Protocol Fee Escapes Its Stated Subordination Inside The Termination Epoch	MEDIUM	Missing Post-Operation Health Check (SCWE-125)	✓ FIXED
M006	fundRedemptions() Retires Principal At The Pool Average, Causing The Capacity Cap To Drift From Actual Principal	MEDIUM	Missing Supply Cap Enforcement (SCWE-116)	✓ FIXED
M007	Deposit And Borrow Gates Omit Scheduled Redemption Liabilities, Letting New Lender Capital Fund Queued Exits	MEDIUM	Missing Post-Operation Health Check (SCWE-125)	ACKNOWLEDGED
M008	Interest Compounds Per Segment, So Realised APR Exceeds The Configured Rate And Is Poke-Dependent	MEDIUM	Business Logic Error (SC02-LogicErrors)	✓ FIXED
M009	A Lender Blacklisted By The Loan Asset Cannot Be Paid, Stranding Their Recovery Slice As Dead Capital	MEDIUM	Transfers to Addresses That Cannot Receive Funds (SCWE-149)	✓ FIXED
M010	The Delinquency Penalty Is Charged On The Whole Pool, So A One-Unit Shortfall Bills Penalty APR On Every Lender's Principal	MEDIUM	Failure to Handle Edge Cases (SCWE-083)	ACKNOWLEDGED
M011	Withdrawal Pause Removes The Only Cure For A Matured Redemption Cohort While The Unpausable Penalty Machine Keeps Billing The Whole Pool	MEDIUM	Missing Post-Operation Health Check (SCWE-125)	✓ FIXED
L001	Funded-Lender Redemptions Can Be Frozen After Termination	LOW	Denial of Service (SCWE-087)	✓ FIXED
L002	`redeem()` Does Not Enforce the Registry Freeze That `claimRecovery()` Enforces	LOW	Access Control (SCWE-016)	✓ FIXED

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
L003	Terminated Vaults Permanently Lock Cash Held Above The Frozen Book	LOW	Ether Locked Due to Missing Withdrawal Path (SCWE-140)	✓ FIXED
L004	ERC-4626 max* Views Contradict Execution	LOW	Failure to Handle Edge Cases (SCWE-083)	✓ FIXED
L005	Deposit Gate Validates The Receiver But Never The Payer	LOW	Insufficient Authorization Checks (SCWE-016)	ACKNOWLEDGED
L006	Unbounded Per-Epoch Accrual Loop Scales Gas With Elapsed Time	LOW	DoS with Block Gas Limit (SCWE-058)	ACKNOWLEDGED

17 findings across the assessment.

Findings are ordered by severity.

M001 · BUSINESS LOGIC (SC02-LOGICERRORS)

MEDIUM

FIXED

`repay()` Pulls the Full Amount With No Refund of Overpayment

DESCRIPTION

`repay(amount)` pulls the entire `amount` from the caller and clamps `borrowedAmount` at zero, but never refunds the portion that exceeds the borrower's actual obligation (`amountOwed()` / `_fundingDeficit`), and there is no cap of `amount` at the current obligation nor any refund path in `_settleRepayment` . The surplus simply remains in the vault with no mechanism to return it. Over-repayment is not purely a fat-finger case: because `amountOwed()` grows with interest between the moment it is read and the moment the transaction executes, a borrower attempting a full payoff through `repay()` must send a buffer above the last-read figure to avoid under-paying and that buffer becomes unrecoverable. Once paid in, the excess cannot be retrieved directly: pre-termination it is recoverable only by calling `borrow()` again (which itself reverts while the vault is delinquent, paused, or terminated), and once the vault is wound down the surplus above the frozen lender book is swept to the fee recipient / distributed to lenders via the recovery pool rather than returned to the borrower. `repayAll()` avoids the problem by computing the exact deficit, but `repay(amount)` offers no equivalent protection and no safeguard against permanent loss.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultBorrower.sol#L65
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultBorrower.sol#L156

BUSINESS IMPACT

An overpaying borrower has the excess stranded in the vault with no direct refund; it is recoverable only by re-borrowing (unavailable when the vault is delinquent, paused, or terminated) and is permanently lost to the borrower once wind-down captures it for fees and lenders.

RECOMMENDED NEXT STEP

Cap the pulled amount at the current obligation e.g. pull `min(amount, amountOwed())` or refund `amount - owed` to `msg.sender` inside `_settleRepayment` ; at minimum, document that callers must use `repayAll()` for a full payoff and that any surplus paid via `repay()` is non-refundable.

RETEST

Fixed. _settleRepayment now clamps the pull to the current obligation (amountOwed()) and reverts when nothing is owed, so an over-payment can no longer be stranded in the vault; the new sweepResidual() additionally returns post-cutoff excess cash to the borrower.

M002 · PRIVILEGED ROLE MISMANAGEMENT (SCWE-017)

MEDIUM

ACKNOWLEDGED

Lender Revocation Permanently Strands That Lender's Recovery Slice As Dead Capital

DESCRIPTION

Any lender who is revoked and still holds unfunded shares (wallet-held or escrowed) when the vault terminates has their recovery slice permanently stranded regardless of whether the revocation happened before or after termination, because the approval check is enforced at claim time, not at revocation time. `claimRecovery()` re-checks `lenderRegistry.isApproved(msg.sender)` on every call, and `canTransfer()` returns `false` unconditionally once `terminatedAt != 0`. Such a lender therefore has no exit at all the approval check reverts before `_returnEscrowedShares` can hand their escrow back, and the transfer freeze prevents selling to an approved holder. Because `_snapshotResolution()` freezes `resolutionShares` at the full `totalSupply()`, and every claim is computed as `units * pool / resolutionShares`, that lender's proportional slice of the recovery pool is never assigned to anyone: the remaining lenders each draw exactly their own `units_i / resolutionShares` share, and the revoked lender's portion stays in the contract with no path out. `_feeSweepable()` also subtracts the full lender book (including the revoked shares), so the fee sweep cannot reach it either. Revocation is available to `LENDER_APPROVER_ROLE`, which `RoleManager` documents as a hot ops key holding "freeze powers only it cannot approve". The stranding is avoided only if the revoked lender exits before termination either by having their request funded and then calling `redeem()` (which performs no approval re-check, see #M003) or, while `transfersEnabled` is set, by transferring their shares to an approved holder before the wallet-to-wallet freeze takes effect.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L296
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L326
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L171

BUSINESS IMPACT

The revoked lender's pro-rata slice of the recovery pool becomes permanently locked dead capital unclaimable by the revoked lender, not redistributed to the remaining lenders, and not sweepable as fees. The remaining lenders are not diluted; each still receives their exact fair share, so the loss is the stranded slice itself sitting unreachable in the vault for the life of the contract. A compromised approver key reproduces this across every terminated vault at once.

RECOMMENDED NEXT STEP

Renormalise `resolutionShares` to exclude a permanently-forfeited holder so the remaining lenders absorb the slice, or exempt `claimRecovery()` from the registry check post-termination and enforce compliance at the payout destination instead; either way add an explicit forfeiture / skim path for a slice that can never be claimed.

RETEST

Acknowledged. A revoked lender's recovery is frozen, not forfeited never redistributed, never swept, and thaws in full on re-approval. Both remedies in the report would either hand a sanctioned party's property to other lenders or pay the sanctioned address directly.

M003 · TRANSFERS TO ADDRESSES THAT CANNOT RECEIVE FUNDS (SCWE-149)

MEDIUM

redeem() Pays Only msg.sender, Permanently Stranding Claims And Vault Liquidity For Non-Receiving Lenders

✓ FIXED

DESCRIPTION

`redeem()` takes no arguments and pays via `_payUsdc(msg.sender, assets)`, so a lender cannot nominate an alternative recipient. Both ERC-4626 and ERC-7540 specify a `receiver` parameter for exactly this reason. The vault is intended for USDC, which enforces an issuer-controlled blocklist. If a lender's address is blocked after their redemption is funded, the transfer reverts on every attempt and `reservedAssets` is never decremented. Because `_cashForClaims()` subtracts `reservedAssets` and `_requiredVaultBalance()` adds it, the stranded amount is permanently removed from `_freeLiquidity()` as well, so it cannot be borrowed, swept, or recovered by anyone.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L242-L268
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L492-L497

BUSINESS IMPACT

The affected lender's settled payout is permanently unrecoverable, and the same amount is permanently subtracted from the borrower's drawable liquidity and from the fee sweep ceiling for the life of the vault. In testing, one stranded claim reduced `availableToBorrow()` from 1,200,000 to 197,807 with no path to restore it. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Add a `receiver` parameter to `redeem()` in line with ERC-4626/ERC-7540, and provide an administrative path to re-target or write off a claim that can no longer be delivered.

RETEST

Fixed. A redeem(address receiver) overload was added; the payout is sent to a caller-nominated receiver while the entitlement and approval remain keyed to msg.sender, so a payout address that cannot receive funds no longer strands the claim.

M004 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

MEDIUM

fundRedemptions() Leaves The Delinquency Cash Watermark Stale, Disabling The Anti-Donation Clamp

✓ FIXED

DESCRIPTION

`lastAccrualCash` is documented in `LoanVaultDeposits` as the watermark that makes a past shortfall unbuyable: past epoch boundaries are tested against `min(live cash, watermark)` so that "an untracked donation cannot raise it, so a bare ERC20 transfer cannot buy back a shortfall that already happened." The watermark is refreshed only inside `_pullUsdc`, `_payUsdc` and at the end of `_accrueInterest`. `fundRedemptions()` increments `reservedAssets` by `need` without moving any cash and without re-stamping the watermark, which lowers `_cashForClaims()` by `need` while leaving `lastAccrualCash` at its pre-funding value. The clamp in `_maybeAnchorDelinquency` then selects live cash rather than the watermark, and live cash includes any untracked donation precisely the input the clamp exists to exclude. `_accrueInterest` also returns early when `last == block.timestamp`, before reaching the re-stamp, so a same-block `update()` does not close the window.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L228
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L475-L484
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L332-L333

BUSINESS IMPACT

A borrower who is short at an epoch boundary can suppress the retroactive delinquency anchor with a bare ERC-20 transfer instead of `repay()`, avoiding a penalty that would otherwise accrue to lenders' share price. The dodge is silent: `_maybeAnchorDelinquency` returns before its `DelinquencyStarted` emit, so no event records the missed boundary. The creditable amount is bounded by the size of the last funding round, and the window closes on the next accrual-completing call (`update()` is permissionless), so exploitation requires an un-poked window between funding and the shortfall boundary. The likelihood of exploitation is medium.

RECOMMENDED NEXT STEP

Re-stamp `s.lastAccrualCash = _cashForClaims(s)` at the end of `fundRedemptions()` so the watermark reflects the post-funding cash. (The report's secondary suggestion to move the stamp ahead of the `last == block.timestamp` early return should be reviewed carefully, as re-stamping to live cash can itself bake a donation into the watermark.)

RETEST

Fixed. A _stampCashWatermark() call was added after every movement of reservedAssets (in fundRedemptions and in redeem, including the zero-payout path), so lastAccrualCash can no longer be left stale-high and the anti-donation clamp on the retroactive delinquency anchor holds.

M005 · MISSING POST-OPERATION HEALTH CHECK (SCWE-125)

MEDIUM

✓ FIXED

Protocol Fee Escapes Its Stated Subordination Inside The Termination Epoch

DESCRIPTION

`collectFees()` is documented as "SUBORDINATE, NOT FORFEIT … only out of what is left once every outstanding lender entitlement is covered", and `_feeSweepable()` implements that by subtracting the lender book from available cash. Before the accrual cutoff, `resolutionShares` is necessarily zero `_snapshotResolution` is reachable only from `claimRecovery()`, which reverts below the cutoff so `_feeSweepable` takes the `ds.totalAssets` branch. Its comment asserts that pool is "already frozen", but `terminate()` explicitly keeps interest accruing through the end of the termination epoch. The guard is therefore measured against a book value that is still growing, and a sweep executed inside that window is sized against a smaller lender entitlement than the one `frozenAssets` will record at the cutoff. `collectFees()` has no cutoff gate, so the manager can sweep in this window while the lender book is understated. The harm is realised only when held cash sits between the at-terminate book and the final frozen book: below that range `_feeSweepable` is already zero (nothing to sweep), and above it the fee is genuine surplus.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L402-L410
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L305
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L327

BUSINESS IMPACT

Fees swept between `terminate()` and the accrual cutoff take value that the wind-down is about to book as a lender entitlement, capped at one termination epoch of net base interest on the whole pool. Because `claimRecovery()` caps the pool at `frozenAssets` and no fee-return path exists, the shortfall is permanent and falls on all remaining lenders pro rata. The fee recipient is manager-settable via `setFeeRecipient`, so the swept value can be directed to the manager's own address. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Gate `collectFees()` on `block.timestamp >= _accrualCutoff(ds)` once terminated (e.g. `if (ds.terminatedAt != 0 && block.timestamp < _accrualCutoff(ds)) revert InsufficientFunds();`), so the lender book is final before the subordination arithmetic runs; or value `lenderBook` at the pool projected to the cutoff rather than at live `ds.totalAssets`.

RETEST

Fixed. _feeSweepable() now returns 0 while terminatedAt != 0 && block.timestamp < _accrualCutoff, deferring any fee sweep until the lender book is final; sweepResidual() applies the same cutoff gate, so the fee can no longer jump ahead of interest still accruing to lenders.

M006 · MISSING SUPPLY CAP ENFORCEMENT (SCWE-116)

MEDIUM

FIXED

fundRedemptions() Retires Principal At The Pool Average, Causing The Capacity Cap To Drift From Actual Principal

DESCRIPTION

`ds.principal` is documented as "original deposit amounts, excluding interest" and is the sole input to `_checkCapacity()`, the only on-chain limit on the deposit leg. On entry, `_enterPool()` adds the full deposited amount, which for a late depositor already reflects the premium paid for a share price above par. On exit, `fundRedemptions()` retires `mulDiv(shares, ds.principal, supplyBefore)` the pool average per share rather than the exiting lender's contributed principal. Because interest changes `totalAssets` (and therefore the payout) but changes neither `ds.principal` nor the share supply, the retirement amount is independent of interest, yet the amount added on entry was not: the two attributions disagree whenever the share price differs from par, so `ds.principal` drifts from the sum of outstanding original deposits in either direction depending on entry and exit ordering, and the drift compounds across funding rounds. Worked example: cap 1000; A deposits 100 (100 shares at price 1); interest doubles the pool (price 2); B deposits 800 (400 shares); `ds.principal` = 900, supply = 500. B redeems all 400 shares and is paid 800 at par, while the counter is retired by `mulDiv(400, 900, 500) = 720`, leaving `ds.principal` = 180 though the only remaining lender (A) contributed just 100 80 units of phantom principal in a single break-even cycle, repeatable to ratchet the counter toward the cap.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L222-L227
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L744-L746
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L134-L138

BUSINESS IMPACT

The borrower's authorised exposure limit stops meaning what it says: an unprivileged lender can, through break-even deposit/redeem cycles, drive `ds.principal` up until legitimate deposits are rejected with `ExceedsCapacity` while actual principal sits well below the cap (deposit denial-of-service), or drive it down so that real outstanding principal exceeds the manager-set cap (borrower over-exposure). No pricing, funding, delinquency or wind-down path reads `ds.principal`, so solvency and payouts are unaffected and no funds are lost. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Track principal per exiting share against contributed principal rather than the pool average, or derive available capacity from live pool value (`totalAssets` /supply) rather than the drifting `ds.principal` accumulator. If the approximation is acceptable, document `ds.principal` as a pool-average figure and reflect that in how `vaultCapacity` is set.

RETEST

Fixed. The principal register is deleted entirely storage field and principal() view both gone. Capacity is now an exposure cap tested against the post-accrual pool accrued interest consumes capacity, and maxDeposit reports the cap minus the projected pool, which equals the gate deposit() itself applies.

M007 · MISSING POST-OPERATION HEALTH CHECK (SCWE-125)

MEDIUM

ACKNOWLEDGED

Deposit And Borrow Gates Omit Scheduled Redemption Liabilities, Letting New Lender Capital Fund Queued Exits

DESCRIPTION

`_requiredVaultBalance()` reserves only `reservedAssets` plus the liability of shares that have already matured. Redemption requests that are queued but have not yet crossed their epoch boundary reserve nothing, so `_freeLiquidity()` and therefore `availableToBorrow()`, `borrow()` and the live branch of `_feeSweepable()` treat cash that is already committed to a queued exit as drawable. On the entry side, `_enterPool()` blocks deposits only when the vault is *already* delinquent, which by definition requires matured shares; `maxDeposit()` applies the same test. The window between `requestRedeem()` and the epoch boundary is therefore unguarded on both sides. No view in the contract aggregates the forward redemption schedule, so a prospective depositor has no on-chain signal of pending exit pressure.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L57-L69
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L131
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L226-L242

BUSINESS IMPACT

A lender who queues an exit ahead of a credit event can be repaid at full book value out of capital deposited afterwards, while the later depositor is left holding shares backed only by borrower debt. In testing, a vault holding zero cash with 100% of its pool queued to exit still advertised full remaining capacity via `maxDeposit()`; the subsequent depositor retained 8.9% of their contribution as cash while `convertToAssets()` reported full par. The harm requires the coincidence of a queued exit, a fresh deposit funding it, and a subsequent borrower default, and the later depositor is a voluntary participant taking credit exposure, so the practical likelihood is low.

RECOMMENDED NEXT STEP

Net the full forward `scheduledWithdrawalShares` liability out of `_requiredVaultBalance()`, and expose an aggregate pending-redemption view so depositors can observe queued exit pressure before entering. Note that reserving all forward-scheduled shares also reduces the borrower's drawable liquidity, so weigh reserving-on-request against a deposit-time solvency check plus a pending-exit view.

RETEST

Acknowledged. Deposits may fund queued exits by design. The false NatSpec has been corrected, and the fairness property a late depositor and a never-queued holder recover identically per share is pinned instead of changing the mechanism.

M008 · BUSINESS LOGIC ERROR (SC02-LOGICERRORS)

MEDIUM

Interest Compounds Per Segment, So Realised APR Exceeds The Configured Rate And Is Poke-Dependent

✓ FIXED

DESCRIPTION

`VaultAccountingLib.calculateInterest` is documented as "Simple (non-compounding)", and `_accrueInterest` claims that "timing of the poke never changes the result". Neither holds at the system level. `_accrueInterest` prices each accrual segment against the running pool `a.assets` (`_segmentInterest(s, a.assets, a.segStart, accrueEnd)`), then grows the pool by the net interest (`a.assets += segInt - segFee`), so the next segment is priced against the enlarged pool. Because the loop cuts a new segment at every epoch boundary (`segmentEnd = min(block.timestamp, nextEpochStart)`), interest compounds at least once per epoch even with no pokes; and because `s.lastAccrualTimestamp = block.timestamp` makes each poke a fresh segment boundary, more frequent poking inserts additional compounding points approaching continuous compounding. The realised return therefore exceeds the configured simple rate, by an amount that grows with the rate, and is path-dependent on how often the permissionless `update()` (or any mutator) is called. Verified against the code: the compounding is the `a.assets` -based segment pricing at `LoanVaultDeposits.sol:381` and `:384` ; the poke-dependence follows from the segment boundary being seeded at `lastAccrualTimestamp` .

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L381
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L384
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/VaultAccountingLib.sol#L23

BUSINESS IMPACT

The borrower pays more interest than the configured rate implies, and lenders earn more than the stated simple rate, contradicting the documented "simple, non-compounding" semantics. The overcharge scales with the rate about +6% of interest at 1200 bps but +69% at the 10000 bps (`MAX_INTEREST_RATE_BPS`) cap. Separately, because each poke adds a compounding point, any unprivileged party can inflate the borrower's realised debt (and lenders' yield) by calling `update()` frequently; the manipulable band is bounded by continuous compounding small at low rates (~150 USDC/yr on a 1M pool at 1200 bps) but material near the cap (~25,000 USDC/yr at 10000 bps). The poke-dependence is a definitive violation of the code's stated poke-independence regardless of whether epoch compounding was intended.

RECOMMENDED NEXT STEP

Decide the intended semantics and align code and docs. If simple interest was intended, price every segment against a fixed base (the pre-accrual pool or `principal`) instead of the running `a.assets` , so neither epoch stepping nor poking compounds. If compounding is intended, correct the "Simple (non-compounding)" and "poke timing never changes the result" claims, and pin the compounding frequency to epoch boundaries only (compute each epoch's interest against the pool snapshot at that epoch's start, independent of intra-epoch poke times) so the realised rate is deterministic and not manipulable by poke frequency.

RETEST

Fixed. Interest was re-implemented on a base-period model: each leg bills cumulative-floor differences against a base frozen at the period start, and the period re-bases only at epoch boundaries and genuine pool cash events. Realised interest is now deterministic and independent of poke frequency, and the "simple, non-compounding / poke-independent" documentation was corrected to match.

M009 · TRANSFERS TO ADDRESSES THAT CANNOT RECEIVE FUNDS (SCWE-149)

MEDIUM

A Lender Blacklisted By The Loan Asset Cannot Be Paid, Stranding Their Recovery Slice As Dead Capital

✓ FIXED

DESCRIPTION

Both payout paths pay msg.sender with no receiver argument: redeem() calls _payUsdc(msg.sender, assets) and claimRecovery() calls _payUsdc(msg.sender, pay), each resolving to SafeERC20.safeTransfer. USDC's transfer reverts for a blacklisted recipient, so both calls revert in full for that lender, and there is no receiver parameter with which they could nominate a clean payout address. canTransfer() returns false unconditionally once terminatedAt != 0, so the position cannot be reassigned to an unblocked wallet either. Because _snapshotResolution() freezes resolutionShares at the full totalSupply() and never renormalises it, the blocked holder's shares remain in the recovery denominator; their proportional slice, units_blocked * pool / resolutionShares, is simply never drawn and stays in the vault. _feeSweepable()'s terminated branch returns cash - outstanding where outstanding = frozenAssets - totalRecoveryPaid; since the blocked holder never claims, their slice remains inside outstanding, so cash <= outstanding and the fee sweep evaluates to zero the amount is unreachable by the fee recipient as well. The trigger is the loan-asset issuer's blocklist, entirely outside the protocol, so it requires no privileged action and no compromised key. A registry- or approval-based fix does not help here, since a blocklisted address remains unpayable regardless of its approval status.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L314
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L265
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L322-L329
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L402-L410
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L171

BUSINESS IMPACT

The blacklisted lender permanently loses their entire position: they cannot be paid through redeem() or claimRecovery(), cannot transfer to a clean wallet post-termination, and no rescue or skim path exists anywhere in the contract. Their pro-rata slice of the recovery pool becomes dead capital unclaimable by them, not redistributed to the remaining lenders, and not reachable by the fee sweep. The remaining lenders are not diluted: each still draws exactly their own units_i / resolutionShares share of the pool. The loss is therefore borne by the blacklisted lender (a legitimately-approved party frozen out by an external event), plus the corresponding cash sitting permanently unreachable in the vault. On a two-lender vault with a 2,000,000 USDC recovery pool where one lender holds half the shares, exactly 1,000,000 USDC is permanently dead. For a product whose lender base is KYC- and sanctions-gated, a lender being blocklisted by the asset issuer mid-life is a foreseeable event.

RECOMMENDED NEXT STEP

Add a receiver parameter to redeem() and claimRecovery() so a blocked holder can nominate a clean payout address, and provide a mechanism to renormalise resolutionShares (or otherwise release the residual) when a holder is provably unable to claim, so the stranded cash is not permanently dead.

RETEST

Fixed. The receiver parameter added to redeem() and claimRecovery() lets a lender blocked by the loan asset direct their payout to a clean address, so their recovery slice is claimed rather than stranded for the remaining lenders.

M010 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

MEDIUM

ACKNOWLEDGED

The Delinquency Penalty Is Charged On The Whole Pool, So A One-Unit Shortfall Bills Penalty APR On Every Lender's Principal

DESCRIPTION

`_segmentInterestFrom()` charges `penaltyRateBps` on `exposed` the entire active pool for any segment past the penalty start, and `_maybeAnchorDelinquency()` sets that anchor on any shortfall regardless of size. `requestRedeem()` enforces no minimum share amount, so a single unit of shares can constitute the matured cohort. A borrower in the ordinary post-`borrow()` state holds no idle cash, because `_freeLiquidity()` lets them draw everything not already reserved, so the shortfall test `_cashForClaims < _maturedLiability` is satisfied by a one-unit claim. The result is that the penalty bears no relation to the size of the miss: the base is the pool, not the unfunded liability. The penalty is also credited to the whole pool pro-rata, so it does not compensate the lender who was made to wait that lender collects only their own share of it while every non-exiting lender free-rides.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L421-L457
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L466-L490
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L93-L94
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultBorrower.sol#L49
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L495-L499

BUSINESS IMPACT

In testing, a 10,000,000 USDC vault at 12% base, 20% penalty and a one-day grace period accrued 301,006.062367 USDC over 90 days with no redemption request outstanding, and 823,948.293709 USDC over the same 90 days after a single one-unit `requestRedeem` an extra 522,942.231342 USDC, or 522 basis points of the pool, billed for a shortfall of one base unit. Any approved lender holding one share can trigger it at the cost of gas. The borrower can cure cheaply if they are watching, but the anchor is retroactive to the epoch boundary and the grace period is capped at seven days, so a borrower who is late by more than the grace window pays the full retroactive penalty. The likelihood of exploitation is medium.

RECOMMENDED NEXT STEP

Scale the penalty base to the unfunded liability rather than the whole pool, and route the penalty to the lenders whose claims were actually late instead of crediting it pro-rata to every share.

RETEST

Acknowledged. The size-independent penalty stands by design: it is a covenant charge for missing a payment date, not damages proportional to the miss. The borrower is given multiple notices to fulfil the repayment dues via automated and manual emails.

M011 · MISSING POST-OPERATION HEALTH CHECK (SCWE-125)

MEDIUM

FIXED

Withdrawal Pause Removes The Only Cure For A Matured Redemption Cohort While The Unpausable Penalty Machine Keeps Billing The Whole Pool

DESCRIPTION

`fundRedemptions()` carries `whenWithdrawalsNotPaused` and is the only pre-cutoff writer of `ds.maturedRedeemShares = 0`; the other writer, inside `_returnEscrowedShares()`, is reachable only from `claimRecovery()` and therefore only once `terminatedAt != 0` and the accrual cutoff has passed. The penalty machine has no equivalent gate: `update()` carries no modifier at all and `repay()` carries only `nonReentrant`, and both run `_accrueInterest()`, whose penalty leg is charged on the entire active pool. Three consequences compose while the pause is set. `_refreshDelinquency()` deletes the per-cohort grace grant only when `maturedRedeemShares == 0`, so the pause makes `penaltyFromTimestamp` undeletable and every re-onset for the rest of the pause is penalised from its first instant with no grace. `borrow()` is also pause-gated, so cash the borrower wires in to cure the shortfall cannot be recovered until someone unpauses, and is locked permanently if the manager then calls `terminate()`. `_enterPool()` refuses deposits while `_isDelinquent()`, so the vault cannot raise capital during the same window. Separately, `maxDeposit()` never consults `withdrawalsPaused()`, so once the borrower cures it advertises full remaining capacity on a vault whose every exit path is frozen. This is not H001, which concerns already-funded claims and requires `terminate()`; this concerns the unfunded matured cohort and the borrower side, and needs no termination.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L188
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L229
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L553
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L607
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultBorrower.sol#L40
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L226-L240

BUSINESS IMPACT

For the full duration of the pause the borrower is billed up to `penaltyRateBps` (maximum 5000) on the whole active pool unless they permanently lock cash equal to the matured liability into a vault they cannot draw from. On a 10,000,000 USDC pool at 2000 bps that is roughly 5,479 USDC per day, or 164,000 USDC across a 30-day pause; at the maximum rate it is roughly 13,700 USDC per day. The cure capital is irrecoverable until unpause and permanently irrecoverable if the vault is then terminated. Simultaneously the vault cannot accept deposits while short, and once cured `maxDeposit()` invites new lenders into a position they provably cannot leave, with no ERC-4626 signal. Pausing withdrawals during a credit event is the documented operational response, not an attack. The likelihood of exploitation is medium.

RECOMMENDED NEXT STEP

Exempt `fundRedemptions()` from the withdrawals pause it only ever improves the lender position or suspend penalty accrual for any window in which the pause blocks the borrower's cure. Additionally, return zero from `maxDeposit()` while withdrawals are paused.

RETEST

Fixed. fundRedemptions() was exempted from the withdrawals pause, maxDeposit() and availableToBorrow() now return 0 while withdrawals are paused, and the per-cohort grace grant is released on a cure delivered under the pause, so the cure path is restored during a pause.

L001 · DENIAL OF SERVICE (SCWE-087)

Funded-Lender Redemptions Can Be Frozen After Termination

LOW

✓ FIXED

DESCRIPTION

Once a redemption request is funded via `fundRedemptions()`, the lender's shares are burned and their payout is set aside in `reservedAssets`; the only way to withdraw it is `redeem()`. However, `redeem()` carries the `whenWithdrawalsNotPaused` modifier, whereas `claimRecovery()` is deliberately left non-pausable precisely because a post-termination exit must never be subject to an unbounded freeze. Because a funded lender's shares are already burned, `claimRecovery()` pays them nothing (it pays on share balance), so if the vault manager pauses withdrawals after termination, these lenders have no available exit and their settled funds remain frozen until the manager unpauses.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L242
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L289

BUSINESS IMPACT

Already-funded lenders can be indefinitely blocked from withdrawing their ring-fenced payout after termination, contradicting the contract's own guarantee that post-termination exits are never freezable. Funds are not at risk of loss, only of temporary freeze at manager discretion.

RECOMMENDED NEXT STEP

Exempt already-funded `redeem()` claims from the withdrawals pause once the vault is terminated, mirroring the deliberate non-pausability of `claimRecovery()`.

RETEST

Fixed. redeem() now drops the whenWithdrawalsNotPaused gate once terminatedAt != 0, so a funded lender can always reach their ring-fenced cash on a terminated vault even while withdrawals are paused.

L002 · ACCESS CONTROL (SCWE-016)

LOW

`redeem()` Does Not Enforce the Registry Freeze That `claimRecovery()` Enforces

✓ FIXED

DESCRIPTION

Lender approval is validated at request time in `requestRedeem()` and re-validated on every call in `claimRecovery()`, whose intent is that a revoked lender stays frozen out for as long as the revocation stands. The `redeem()` function performs no `isApproved(msg.sender)` check. Because `fundRedemptions()` is permissionless and can execute after a lender is revoked, a lender who was approved when requesting but revoked before funding is still paid in full through `redeem()`. The same lender calling `claimRecovery()` would be rejected, so the registry revocation is enforced inconsistently across the two exit paths.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L242
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L296

BUSINESS IMPACT

A revoked or sanctioned lender can still extract their funded redemption, defeating the freeze the registry is meant to guarantee. No other user's funds are lost; the impact is a compliance-control bypass, relevant where a revoked address must never receive funds.

RECOMMENDED NEXT STEP

Add an `isApproved(msg.sender)` check to `redeem()` to match `claimRecovery()`; alternatively, if payout to a revoked lender is intended, remove the per-call check from `claimRecovery()` so both exits behave consistently.

RETEST

Fixed. redeem() now re-checks lender approval at payout, mirroring claimRecovery(). A revoked lender's funded payout waits in reservedAssets (non-forfeiting) and is released in full on re-approval, so the two exits enforce the registry freeze consistently.

L003 · ETHER LOCKED DUE TO MISSING WITHDRAWAL PATH (SCWE-140)

Terminated Vaults Permanently Lock Cash Held Above The Frozen Book

LOW

✓ FIXED

DESCRIPTION

After termination the recovery pool is capped at `frozenAssets`, and `_feeSweepable()` allows the fee recipient to draw only up to `accruedFees`. There is no `skim` function and no borrower refund path anywhere in `LoanVaultBorrower`. Cash held above `frozenAssets + reservedAssets + accruedFees` arising from an over-repayment, a direct token transfer, or a borrower who pays more than the frozen book is therefore unreachable by lenders, by the borrower and by the protocol. `repay()` and `repayAll()` accept any amount without an upper bound, so an over-payment during wind-down produces exactly this state.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L304-L305
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L402-L410
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultBorrower.sol#L65-L69

BUSINESS IMPACT

Any surplus a borrower pays into a terminated vault beyond the frozen book is permanently locked; the portion within `accruedFees` is redirected to the fee recipient rather than returned. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Add a permissioned sweep that returns residual cash above all outstanding entitlements to the borrower, or clamp `repay()` at `amountOwed()` once terminated.

RETEST

Fixed. The new sweepResidual() (vault-manager only, post-cutoff, paying the borrower, and strictly junior to both lenders and the fee) returns residual cash held above the frozen book, and the repay() clamp prevents the surplus from arising in the first place.

L004 · FAILURE TO HANDLE EDGE CASES (SCWE-083)

ERC-4626 max* Views Contradict Execution

LOW

✓ FIXED

DESCRIPTION

ERC-4626 requires that `deposit(maxDeposit(x))` and `mint(maxMint(x))` do not revert, and that the `max*` views account for global restrictions. `maxWithdraw()` and `maxRedeem()` delegate straight to `_previewClaim()` with no pause check, while `redeem()` carries `whenWithdrawalsNotPaused` so a paused vault advertises claimable amounts that revert. `maxDeposit()` does honour the deposit pause but returns raw remaining capacity, so when `minimumDepositAmount` is zero a `maxDeposit()` value can round to zero shares and revert `ZeroShares`. `maxMint()` floors to shares and `mint()` re-derives assets with rounding up, so the round trip can land below `minimumDepositAmount` and revert `InvalidDepositAmount`. `availableToBorrow()` and `redemptionFundingShortfall()` are pause-blind in the same way.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L259-L268
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L226-L248
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L308-L310

BUSINESS IMPACT

Integrators following the ERC-4626 contract encounter reverts on values the vault itself advertised as safe, breaking automated deposit and claim flows. No funds are lost. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Have `maxWithdraw` / `maxRedeem` / `availableToBorrow` return zero while the relevant pause is set, and clamp `maxDeposit` / `maxMint` to values the entry leg will actually accept.

RETEST

Fixed. maxWithdraw/maxRedeem return 0 when a redeem would revert; availableToBorrow returns 0 while paused or terminated; maxDeposit guards against a zero-share round-down; and maxMint reports 0 when its ceil-priced cost would fall under the minimum deposit.

L005 · INSUFFICIENT AUTHORIZATION CHECKS (SCWE-016)

Deposit Gate Validates The Receiver But Never The Payer

LOW

ACKNOWLEDGED

DESCRIPTION

`_validateEntry()` checks `lenderRegistry.isApproved(receiver)` only, while `_enterPool()` pulls the assets from `msg.sender`. Any address never approved, or explicitly revoked can therefore fund a deposit credited to an approved lender. For a vault whose lender registry exists to enforce a Master Loan Agreement gate backed by off-chain onboarding, the source of deposited capital is unconstrained even though the recipient of the shares is not.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L118-L122
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVault.sol#L128-L136

BUSINESS IMPACT

Capital from an unvetted or revoked party can enter the vault so long as an approved lender receives the shares, weakening the compliance guarantee the registry is intended to provide. No accounting invariant is broken. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Require `isApproved(msg.sender)` in addition to `isApproved(receiver)`, or document explicitly that third-party funding of an approved lender's position is intended.

RETEST

Acknowledged. Deposits stay gated on the beneficial owner, not the payer: this matches permissioned-pool practice, source-of-funds is unenforceable on-chain, and a payer can extract nothing. The registry documentation that overstated revocation's effect has been corrected.

L006 · DOS WITH BLOCK GAS LIMIT (SCWE-058)

LOW

Unbounded Per-Epoch Accrual Loop Scales Gas With Elapsed Time

ACKNOWLEDGED

DESCRIPTION

`_accrueInterest()` and its view twin `_projectedPoolNow()` both iterate once per elapsed epoch from the last accrual to the current epoch, and `fundRedemptions()` carries a second such loop over `(fundedThroughEpoch, currentEp]`. `MIN_EPOCH_DURATION` is one day and `epochDuration` is fixed at initialisation with no setter, so on a vault configured at the minimum the iteration count is wall-clock elapsed days. Every state-mutating entry point accrues first and therefore inherits the full cost. Measured on a one-day-epoch vault: roughly 1,795 gas per epoch in the views and 1,175 in `update()`, giving about 1.81M gas for a view after one untouched year and 17.94M after ten.

AFFECTED CODE

- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L346
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultDeposits.sol#L290
- https://github.com/Axcess-Finance/axcess-contract/blob/c350e58c4483b70af8784e26a4e15c10dd24c852/src/LoanVaultWithdrawals.sol#L207

BUSINESS IMPACT

A long-dormant vault becomes progressively more expensive to interact with, and its pricing views exceed common `eth_call` gas caps well before the block limit, breaking wallets, indexers and front ends. `update()` is permissionless and free to call, so an active vault is not affected and the dormancy cannot be forced by an attacker. The likelihood of exploitation is low.

RECOMMENDED NEXT STEP

Bound the walk by folding whole untouched epochs arithmetically instead of iterating, or raise `MIN_EPOCH_DURATION` and document a keeper requirement for short-epoch vaults.

RETEST

Acknowledged. The linear catch-up walk stays; at 7-day production epochs the ten-year worst case is ~2.8M gas.

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shreyas Koli
Security Auditor II

Prasad Kuri
Security Auditor

Neel Shah
Security Auditor

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank
Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit
Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR